

# Sistema bancario con TDD

---

Java · JUnit 4 · Herencia · Polimorfismo · Interfaces · Colecciones

## Enunciado

Un banco necesita un sistema para administrar sus cuentas y registrar todo lo que ocurre con ellas. Deben desarrollarlo aplicando TDD con JUnit 4, siguiendo el ciclo Red → Green → Refactor:

1. Se ejecuta el test y falla (Red).
2. Se escribe el código mínimo para que pase (Green).
3. Se mejora el código sin romper los tests (Refactor).

No se puede escribir código de producción sin un test que falle antes.

## Requisitos técnicos

Deben aplicar:

- Interface
- Herencia
- Polimorfismo
- Colecciones
- Enum

## Clases a desarrollar

### Transferible

- boolean depositar(double monto)
- boolean extraer(double monto)
- double getSaldo()

### Cuenta

- Atributos: id (Integer), titular (String), saldo (double).
- depositar: si el monto es  $\leq 0$  devuelve false y no modifica el saldo. Si no, suma el monto y devuelve true.
- extraer: método abstracto.

- Dos cuentas son iguales si tienen el mismo id.

## CajaAhorro

- Cada extracción cobra una comisión fija de \$5.
- extraer devuelve false si el monto es  $\leq 0$  o si  $\text{monto} + 5$  supera el saldo. Si no, descuenta  $\text{monto} + 5$  y devuelve true.

## CuentaCorriente

- Recibe un limiteDescubierto por constructor.
- No cobra comisión.
- extraer devuelve false si el monto es  $\leq 0$  o si el saldo resultante quedaría por debajo de - limiteDescubierto. Si no, descuenta el monto y devuelve true.

## TipoOperacion

- DEPOSITO, EXTRACCION, TRANSFERENCIA\_ENVIADA, TRANSFERENCIA\_RECIBIDA.

## Movimiento

- Atributos: idCuenta (Integer), tipoOperacion (TipoOperacion), monto (double).
- Se crea por constructor y solo tiene getters.

## Banco

- Atributos: nombre, Set<Cuenta> cuentas (**HashSet**) y List<Movimiento> movimientos (**ArrayList**).
- boolean agregarCuenta(Cuenta cuenta)
- Cuenta buscarCuenta(Integer id)
- boolean depositar(Integer id, double monto)
- boolean extraer(Integer id, double monto)
- boolean transferir(Integer idOrigen, Integer idDestino, double monto)
- int cantidadDeCuentas()
- int cantidadDeMovimientos()
- List<Movimiento> getMovimientos()
- List<Movimiento> movimientosDeCuenta(Integer id)

## Reglas del banco

1. Las operaciones devuelven true si se realizaron y false si no.
2. Todas las operaciones se hacen a través del banco, identificando la cuenta por su id.
3. agregarCuenta devuelve false si la cuenta es null o ya existe una con ese id.
4. buscarCuenta devuelve null si la cuenta no existe.
5. Solo las operaciones exitosas generan movimientos. Una operación fallida no modifica nada ni registra nada.
6. depositar registra un movimiento DEPOSITO y extraer uno EXTRACCION, ambos con el monto pedido (sin comisión).
7. transferir devuelve false si el monto es  $\leq 0$ , si origen y destino son la misma cuenta, si alguna cuenta no existe o si el origen no puede extraer.
8. Una transferencia exitosa descuenta del origen (con su comisión, si corresponde), acredita el monto completo al destino y registra dos movimientos: TRANSFERENCIA\_ENVIADA (origen) y TRANSFERENCIA\_RECIBIDA (destino), con el monto transferido. No registra además un DEPOSITO ni una EXTRACCION.
9. Si una transferencia falla, ninguna de las dos cuentas debe quedar modificada.

## Tests a completar

Cada test tiene un fail(...) para que arranque en rojo. Deben reemplazarlo por el test real. Solo después escriben el código de producción.

### BancoTest

```
import static org.junit.Assert.*;

import java.util.List;

import org.junit.Before;

import org.junit.Test;

public class BancoTest {

    private Banco banco;
```

*@Before*

```
public void setUp() {  
  
    banco = new Banco("Banco Escuela");  
  
    banco.agregarCuenta(new CajaAhorro(1, "Ana", 1000));  
  
    banco.agregarCuenta(new CuentaCorriente(2, "Luis", 500, 1000));  
  
}
```

// Depositar 50 devuelve true y el saldo pasa de 100 a 150 en la CajaAhorro.

*@Test*

```
public void depositarCajaAhorroDebeAumentarElSaldo() {  
  
    fail("Completar el test");  
  
}
```

// Extraer 50 de la CajaAhorro devuelve true. Se descuentan 50 + 5 de comisión:

// el saldo final es 45.

*@Test*

```
public void extraerCajaAhorroDebeDescontarMontoMasComision() {  
  
    fail("Completar el test");  
  
}
```

// Extraer 1200 de la CuentaCorriente con id = 2 devuelve true y el saldo queda

// en -700 (usa el descubierto).

*@Test*

```
public void extraerCuentaCorrienteUsandoElDescubiertoDebeDejarSaldoNegativo() {  
  
    fail("Completar el test");  
  
}
```

```
// Se declaran dos variables de tipo Transferible (CajaAhorro y CuentaCorriente,  
// saldo 500). Se llama extraer(100) sobre cada una. La caja queda en 395  
// (comisión) y la corriente en 400.  
  
@Test  
public void extraerSobreTransferibleDebeAplicarLaReglaDeCadaTipoDeCuenta() {  
    fail("Completar el test");  
}
```

```
// Se guardan una CajaAhorro y una CuentaCorriente en un misma coleccion de  
// Cuenta. Se deposita 100 en cada una y se verifica que ambas subieron 100  
  
@Test  
public void depositarEnColeccionDeCuentasDebeTratarATodasPorIgual() {  
    fail("Completar el test");  
}
```

```
// Agregar una CajaAhorro con id nuevo devuelve true y cantidadDeCuentas  
// aumenta.  
  
@Test  
public void agregarCajaAhorroNuevaDebeDevolverTrueYAumentarLaCantidad() {  
    fail("Completar el test");  
}
```

```
// No debería permitir agregar una cuenta con id existente  
  
@Test  
public void noPermitirAgregarCajaDeAhorroConIdDuplicado() {  
    fail("Completar el test");  
}
```

// Buscar una Cuenta Corriente con id 9 retorna null

*@Test*

public void buscarCuentaInexistenteDebeRetornarNull() {

*fail*("Completar el test");

}

// Tras depositar en la CajaAhorro hay 1 movimiento: idCuenta 1, tipo DEPOSITO,

// monto pedido.

*@Test*

public void depositarEnCajaAhorroDebeRegistrarUnMovimientoDeDeposito() {

*fail*("Completar el test");

}

// Tras transferir entre CajaAhorro y CuentaCorriente hay 2 movimientos, en

// orden: TRANSFERENCIA\_ENVIADA y TRANSFERENCIA\_RECIBIDA.

*@Test*

public

void

transferirEntreCajaAhorroYCuentaCorrienteDebeRegistrarMovimientoEnviadoYRecibido() {

*fail*("Completar el test");

}

// transferir con destino inexistente devuelve false. El saldo de la CajaAhorro

// origen no debe descontarse y no hay movimientos.

*@Test*

public void transferirConDestinoInexistenteNoDebeDescontarDeCajaAhorroOrigen() {

*fail*("Completar el test");

```
}
```

```
// Se hace depositar, extraer y transferir. Se verifica que getMovimientos() los
```

```
// devuelve en el orden cronológico en que ocurrieron.
```

```
@Test
```

```
public void depositarExtraerYTransferirDebeMantenerElOrdenCronologicoDeMovimientos() {
```

```
    fail("Completar el test");
```

```
}
```

```
}
```